

01 UART Specification

- [SMC UART 16550 — Functional Specification](#)
 - [Contents](#)
 - [1. Purpose and scope](#)
 - [2. Conformance and references](#)
 - [3. Feature summary](#)
 - [4. Configuration parameters](#)
 - [4.1 CCI parameters \(primary interface\)](#)
 - [4.2 `uart\_cfg\_struct` \(backward-compatibility defaults\)](#)
 - [5. Block diagram and port list](#)
 - [5.1 Block diagram](#)
 - [5.2 Ports](#)
 - [6. Theory of operation](#)
 - [6.1 Operating principle](#)
 - [6.2 Architectural concepts](#)
 - [6.3 Transmit datapath](#)
 - [6.4 Receive datapath](#)
 - [6.5 RX timeout](#)
 - [6.6 Interrupt priority chain](#)
 - [6.7 DMA modes](#)
 - [7. Register map](#)
 - [7.1 Memory map \(DLAB = 0\)](#)
 - [7.2 Memory map \(DLAB = 1, LCR\[7\] = 1\)](#)
 - [7.3 Detailed register descriptions](#)
 - [8. Functional behaviour](#)
 - [8.1 Transmit sequence](#)
 - [8.2 Receive sequence](#)
 - [8.3 FIFO reset behaviour](#)
 - [8.4 Loopback modes](#)
 - [9. Reset behaviour](#)
 - [10. Bus interface \(TLM-2.0 / AXI4-Lite\)](#)
 - [10.1 Transport function](#)
 - [10.2 Error responses](#)
 - [10.3 Debug transport](#)
 - [10.4 DMI](#)
 - [11. Error handling](#)
 - [11.1 RX errors](#)
 - [11.2 Aggregate error output](#)
 - [12. Programming model](#)
 - [12.1 Initialisation sequence](#)
 - [12.2 Transmit a character](#)
 - [12.3 Receive a character](#)
 - [12.4 Interrupt-driven operation](#)
 - [13. Compliance matrix](#)
 - [14. Revision history](#)

- [15. Glossary.](#)

SMC UART 16550 — Functional Specification

Document: 01_UART_Specification.md **Module:** smc::uart (SystemC/TLM-2.0 Loosely-Timed model) **Spec base:** NS16550A / UART 16550 (compatible) **RTL reference:** hw/comp/uart_16550/rtl/ (uart_16550.sv, uart_core.sv, uart_tx.sv, uart_rx.sv) **Status:** Design phase — document governs the planned peripherals/uart/ model **Companion docs:** - 02_UART_LowLevel_Design.md — internal SystemC implementation - 03_UART_Test_Plan.md — verification strategy and test list

Contents

1. [Purpose and scope](#)
 2. [Conformance and references](#)
 3. [Feature summary](#)
 4. [Configuration parameters](#)
 5. [Block diagram and port list](#)
 6. [Theory of operation](#)
 7. [Register map](#)
 8. [Functional behaviour](#)
 9. [Reset behaviour](#)
 10. [Bus interface \(TLM-2.0 / AXI4-Lite\)](#)
 11. [Error handling](#)
 12. [Programming model](#)
 13. [Compliance matrix](#)
 14. [Revision history](#)
 15. [Glossary](#)
-

1. Purpose and scope

The SMC UART is a 16550-compatible **Universal Asynchronous Receiver/Transmitter** peripheral for the SMC chiplet. Multiple independent UART instances (default 4) coexist in the platform; each instance is accessed over AXI4-Lite and exposes a 256-byte memory-mapped register window. The UART is used primarily for console/debug output during firmware bring-up and for low-speed host communication.

This specification defines the **externally-observable behaviour** of one UART instance as implemented by the SystemC/TLM-2.0 loosely-timed functional model: configuration parameters, register map, transmit/receive datapath semantics, interrupt generation, modem control, DMA handshake, reset, and the TLM-2.0 bus contract. It is the contract that:

- Firmware (register-mapped driver code) programs against,
- The RTL (uart_16550.sv and its sub-blocks) implements at gate level, and
- The SystemC model (peripherals/uart/) implements at transaction level.

The model is a **functional** model: register-level behaviour and character-level data flow are reproduced faithfully (and are bit-compatible with the RTL register layout), while bit-level serial timing is abstracted away (see §13).

Internal implementation choices (process topology, FIFO modelling, quantum-keeper usage, etc.) are documented in [02 UART LowLevel Design.md](#).

2. Conformance and references

Source	Authority
hw/comp/uart_16550/data/registers/rdl/uart_16550_main.rdl	Ground-truth register map (DLAB=0 read map)
hw/comp/uart_16550/data/registers/rdl/uart_16550_main_wo.rdl	Ground-truth write-only map (THR, FCR)
hw/comp/uart_16550/data/registers/rdl/uart_16550_dl.rdl	Ground-truth divisor-latch map (DLL, DLM)
hw/comp/uart_16550/rtl/uart_16550.sv	Top-level reference RTL (component)
hw/comp/uart_16550/rtl/uart_core.sv	Functional reference RTL — registers, FIFOs, interrupt/DMA logic
hw/comp/uart_16550/rtl/uart_tx.sv	Transmit-engine reference RTL
hw/comp/uart_16550/rtl/uart_rx.sv	Receive-engine reference RTL
hw/periph/uart_wrap/rtl/templates/uart_wrap.sv.tpl	Platform wrapper integration context (multi-instance)
NS16550A data sheet (National Semiconductor)	Industry-standard UART baseline

The model is **bit-compatible** with the RDL register layout and protocol-compatible with the 16550 programming model. All deliberate deviations (e.g., abstracted baud-rate timing) are listed in §13.

3. Feature summary

Feature	Value / behaviour
Standard	NS16550A compatible
TX FIFO depth	Configurable (default 32 bytes); <code>TX_FIFO_DEPTH</code> CCI param
RX FIFO depth	Configurable (default 32 bytes); <code>RX_FIFO_DEPTH</code> CCI param
FIFO mode	Enabled/disabled per <code>FCR.FIFO_ENABLE</code> ; reset on mode change
Word length	5, 6, 7, or 8 bits (<code>LCR.WLS</code>)
Stop bits	1 or 2 (<code>LCR.STB</code> ; 1.5 stop bits for 5-bit words)
Parity	None, odd, even, stick-0, stick-1 (<code>LCR.PEN</code> / <code>LCR.EPS</code> / <code>LCR.STICK_PARITY</code>)
Break generation	<code>LCR.SET_BREAK</code> forces tx_o low
Baud rate	Programmed via 16-bit divisor (<code>DLL</code> / <code>DLM</code>); disabled when divisor = 0
Loopback	System loopback (TX->RX internal) and line loopback (<code>MCR.LOOP</code> / <code>MCR.LINE_LOOPBACK</code>)
Modem control	CTS, DSR, RI, DCD inputs; RTS, DTR, OUT1, OUT2 outputs
DMA handshake	<code>RXRDY</code> / <code>TXRDY</code> outputs; DMA Mode 0 (single-transfer) and Mode 1 (burst)
Interrupt sources	6: Received-data-ready, THRE, Receiver-line-status, Modem-status, Reception-timeout, FIFO-error
Interrupt priority	Fixed: FIFO-error > Line-status > Timeout > Data-ready > THRE > Modem-status
Bus interface	TLM-2.0 LT target socket (AXI4-Lite, 32-bit data, 32-bit address)
Register window	256 bytes (0x00 .. 0xFF)
DLAB muxing	Address 0x0/0x4 decode to THR/RBR/IER (<code>DLAB=0</code>) or DLL/DLM (<code>DLAB=1</code>)
Reset polarity	Active-low asynchronous (<code>rst_ni</code>)

4. Configuration parameters

4.1 CCI parameters (primary interface)

The model exposes the following OSCI CCI (`cci_configuration`) parameters. These are the **primary** configuration interface; test benches and platform integrators must use the CCI broker to set them.

CCI parameter name	Type	Mutability	Default	Range	Notes
<code>tx_fifo_depth</code>	<code>cci_param<unsigned></code> (<code>CCI_IMMUTABLE_PARAM</code>)	Immutable	32	1..4096	TX FIFO depth in bytes. Maps to <code>TX_FIFO_DEPTH</code> in <code>uart_16550.sv</code> (RTL component default 16; the <code>uart_wrap</code> platform instantiates 32).
<code>rx_fifo_depth</code>	<code>cci_param<unsigned></code> (<code>CCI_IMMUTABLE_PARAM</code>)	Immutable	32	1..4096	RX FIFO depth in bytes. Maps to <code>RX_FIFO_DEPTH</code> in <code>uart_16550.sv</code> (RTL component default 16; <code>uart_wrap</code> instantiates 32).
<code>access_delay_ns</code>	<code>cci_param<double></code>	Mutable	2.0	ns	Annotated delay added to <code>sc_time&</code> delay in <code>b_transport</code> . Approximates AXI4-Lite bus latency.

Hierarchical param names follow the standard CCI convention `<broker_prefix>.<sc_module_name>.<param_name>`, e.g.:

```
// Override before module construction (preset):
broker.set_preset_cci_value("top.uart0.tx_fifo_depth",
                           cci::cci_value::from_json("64"));
broker.set_preset_cci_value("top.uart0.rx_fifo_depth",
                           cci::cci_value::from_json("64"));

// Change access delay at any point (mutable):
auto h = broker.get_param_handle("top.uart0.access_delay_ns");
h.set_cci_value(cci::cci_value(5.0));
```

At construction the UART logs an `SC_REPORT_INFO` showing each parameter's resolved value and whether it came from a CCI preset (`[preset]`) or the constructor default (`[default]`).

4.2 `uart_cfg` struct (backward-compatibility defaults)

The constructor also accepts an optional `uart_cfg cfg` argument whose fields supply the **default values** for the CCI params. A broker preset set before construction takes priority.

```
smc::uart dut("uart0"); // all CCI defaults (32/32/2 ns)
smc::uart dut("uart0", {.tx_fifo_depth = 64,
                        .rx_fifo_depth = 64}); // defaults from cfg; preset can still win
```

`uart_cfg` also holds the fixed address-map constants (not CCI params because they must never vary at run-time):

Constant	Value	Description
WINDOW_SIZE	0x100	256-byte decoded window
REG_WIDTH	4	Register stride (bytes)

5. Block diagram and port list

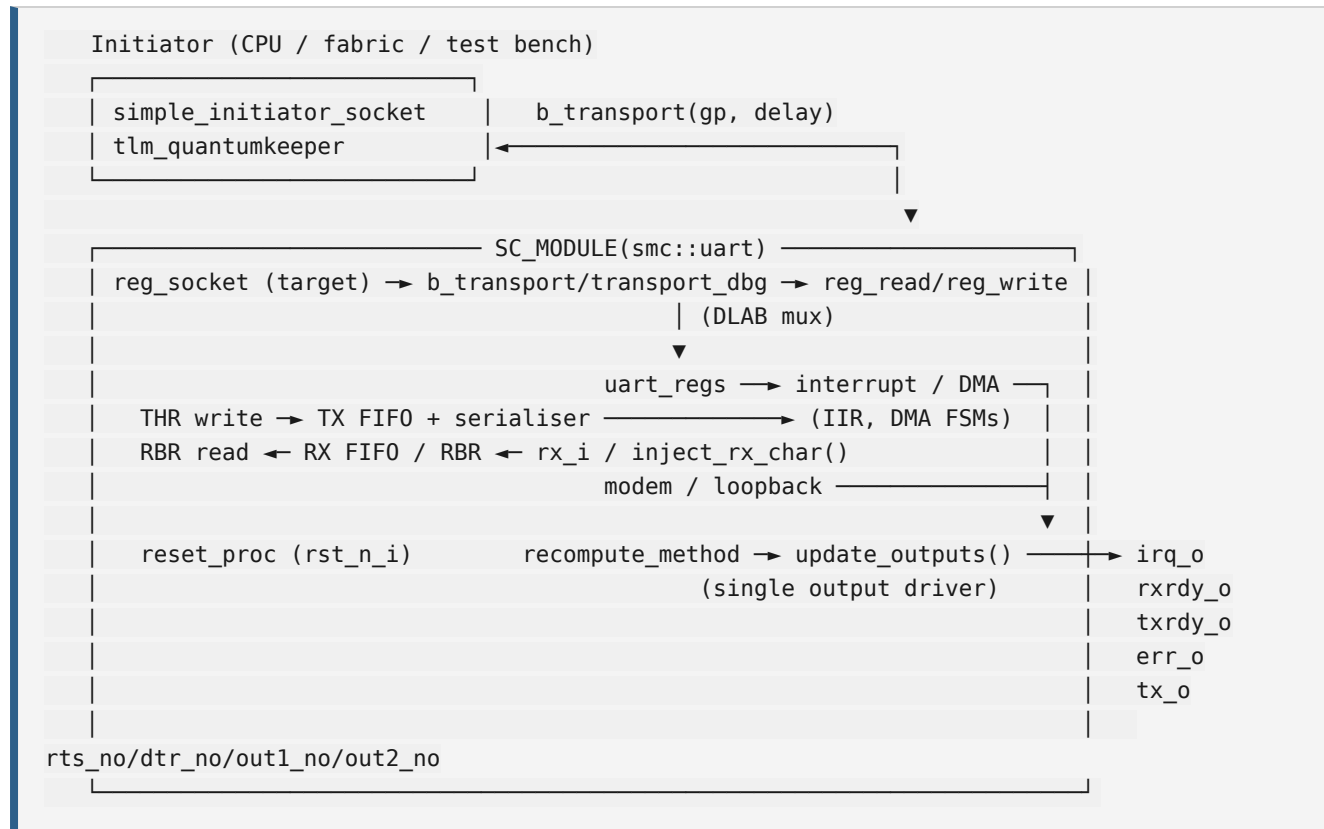
5.1 Block diagram

The diagram below shows the TLM-2.0 sockets, the `smc::uart` internal blocks, the single-output-driver discipline, and the external endpoints each port connects to.

 `smc::uart` socket and module architecture: an initiator (CPU/fabric/test bench) drives a `simple_initiator_socket` through a `tlm_quantumkeeper` into the UART's `reg_socket` target via `b_transport`; inside the module, `reg_read/reg_write` performs the DLAB mux into the `uart_regs` register file, the TX FIFO+serialiser feeds `tx_o`, the RX FIFO/RBR is fed by `rx_i` or the `inject_rx_char` back door, and the interrupt/DMA and modem/loopback blocks plus the register file all converge on a single `update_outputs()` driver that writes `irq_o`, `rxrdy_o`/`txrdy_o`, `err_o`, and the modem outputs; `reset_proc` and `recompute_method` are the `SC_METHOD` processes; external endpoints are the serial peer, modem signals, DMA controller, `smc::plic`, fabric error slave, and reset controller.

`smc::uart` socket and module architecture: an initiator (CPU/fabric/test bench) drives a `simple_initiator_socket` through a `tlm_quantumkeeper` into the UART's `reg_socket` target via `b_transport`; inside the module, `reg_read/reg_write` performs the DLAB mux into the `uart_regs` register file, the TX FIFO+serialiser feeds `tx_o`, the RX FIFO/RBR is fed by `rx_i` or the `inject_rx_char` back door, and the interrupt/DMA and modem/loopback blocks plus the register file all converge on a single `update_outputs()` driver that writes `irq_o`, `rxrdy_o`/`txrdy_o`, `err_o`, and the modem outputs; `reset_proc` and `recompute_method` are the `SC_METHOD` processes; external endpoints are the serial peer, modem signals, DMA controller, `smc::plic`, fabric error slave, and reset controller.

In the SystemC model the serial `tx_o` / `rx_i` lines are not simulated at bit level. TX writes are captured at character granularity; RX injection is done via a back-door method. Modem signal changes are reflected in MSR without baud-rate delay.

ASCII fallback

5.2 Ports

Port	Direction	Type	Description
reg_socket	target	tlm_utils::simple_target_socket<uart>	TLM-2.0 LT register-access socket (AXI4-Lite-style)
rst_n_i	input	sc_in<bool>	Active-low asynchronous reset
tx_o	output	sc_out<bool>	Serial transmit line (idle high)
rx_i	input	sc_in<bool>	Serial receive line (idle high)
cts_ni	input	sc_in<bool>	Clear to Send (active-low)
dsr_ni	input	sc_in<bool>	Data Set Ready (active-low)
ri_ni	input	sc_in<bool>	Ring Indicator (active-low)
dcd_ni	input	sc_in<bool>	Data Carrier Detect (active-low)
rts_no	output	sc_out<bool>	Request to Send (active-low)
dtr_no	output	sc_out<bool>	Data Terminal Ready (active-low)
out1_no	output	sc_out<bool>	User Output 1 (active-low)
out2_no	output	sc_out<bool>	User Output 2 (active-low)
rxrdy_o	output	sc_out<bool>	DMA RX ready
txrdy_o	output	sc_out<bool>	DMA TX ready
err_o	output	sc_out<bool>	Aggregate error flag
irq_o	output	sc_out<bool>	Interrupt output (active-high)

Port names follow the `uart_16550.sv` component top-level. In the `uart_wrap` platform these are exposed per-instance with a `uart_` prefix (e.g. `uart_irq_o[i]`).

6. Theory of operation

6.1 Operating principle

The UART converts parallel register data to and from a serial byte stream:

1. **Transmit:** Software writes a byte to the THR (or TX FIFO). The byte is held until the baud-rate clock fires and the shift register is free, then serialised MSB/LSB with configurable framing (start + data + parity + stop).
2. **Receive:** Incoming serial data is synchronised, majority-filtered, and de-serialised by the receive engine. Completed characters are pushed into the RX FIFO (or RBR in non-FIFO mode) where software reads them.

3. **Interrupt/DMA:** Several conditions — FIFO watermark reached, TX idle, line-status errors, modem-status changes, RX timeout — can assert `irq_o`. DMA handshake signals (`rxrdy_o`, `txrdy_o`) provide an alternative non-interrupt path.

6.2 Architectural concepts

Concept	Definition
THR	Transmitter Holding Register — the write port to the TX FIFO/buffer.
RBR	Receiver Buffer Register — the read port from the RX FIFO/buffer.
TX FIFO / shift reg	Staging queue + serialiser. Depth configurable (default 32).
RX FIFO / RBR	De-serialised character queue. Depth configurable (default 32). Each entry carries the character byte plus <code>break_err</code> , <code>framing_err</code> , <code>parity_err</code> flags.
Baud clock x16	Internal tick at 16× the baud rate; used by both TX and RX engines. Generated from <code>system_clock / (16 * divisor)</code> . Disabled when divisor = 0.
DLAB	Divisor Latch Access Bit (LCR[7]). When set, address 0x0 and 0x4 decode to DLL/DLM instead of THR/RBR/IER.
DMA Mode 0	Single-transfer: RXRDY asserted while RX FIFO is non-empty; TXRDY asserted while TX FIFO is non-full.
DMA Mode 1	Burst-transfer: RXRDY FSM asserts from watermark/timeout until FIFO drains to empty; TXRDY FSM asserts until FIFO fills then deasserts until empty.
Loopback	System loopback (MCR.LOOP): TX output internally feeds RX input; modem outputs driven by modem control bits. Line loopback (MCR.LINE_LOOPBACK): <code>rx_i</code> feeds <code>tx_o</code> ; modem outputs follow modem inputs.

6.3 Transmit datapath

```
THR write → TX FIFO → [baud_x16 tick + fifo-not-empty]
                        → TX serialiser (functional)
                        → frame: start | data[7:0] | [parity] | stop(s)
                        → tx_o (or '0' if SET_BREAK)
```

The TX FIFO is popped and serialised when the serialiser is idle. If `SET_BREAK` is asserted, `tx_o` is forced to 0 regardless of the serialiser state. In the functional model the character is emitted as a whole (with framing computed from LCR); bit-level serial timing is not reproduced.

6.4 Receive datapath

```
rx_i / inject_rx_char() → RX assembler (functional)
                        → framing check: start | data[7:0] | [parity] | stop
                        → {character, framing_err, parity_err, break_err}
                        → RX FIFO push (or RBR write)
```

Received characters are assembled at character granularity. In system loopback mode the RX input is taken from the most recently transmitted character; in line loopback mode the RX input is idle (no characters assembled). Error conditions (parity, framing, break) are supplied by the test bench through the back-door injection method (see §13).

6.5 RX timeout

An RX timeout interrupt fires when at least one character is present in the RX FIFO and no new character has arrived for a period equivalent to $4 \times$ character times. The timeout is cleared by reading all characters from the RX FIFO.

6.6 Interrupt priority chain

Six interrupt sources are encoded into `IIR.INTERRUPT_ID` using a fixed-priority scheme:

Priority	IIR.INTERRUPT_ID	Source
0 (highest)	0x7	FIFO Error
1	0x3	Receiver Line Status (OE / PE / FE / BI)
2	0x6	Reception Timeout
3	0x2	Received Data Ready (RX FIFO watermark)
4	0x1	Transmitter Holding Register Empty
5 (lowest)	0x0	Modem Status (DCTS / DDSR / TERI / DDCD)

`IIR.INTERRUPT_PENDING` is active-low. It reads 0 when any enabled interrupt is active, 1 when none are pending.

6.7 DMA modes

Mode 0 (`FCR.DMA_MODE_SELECT = 0`): RXRDY is high while RX FIFO is non-empty; TXRDY is high while TX FIFO is not full. Used with single-transfer DMA controllers.

Mode 1 (`FCR.DMA_MODE_SELECT = 1`, FIFO must be enabled): RXRDY uses a two-state FSM — it asserts on watermark-reached or timeout, and stays asserted until the FIFO becomes empty. TXRDY uses the complementary FSM — it asserts while the FIFO is not full, and de-asserts when full until the FIFO drains to empty. Used with burst DMA controllers.

7. Register map

7.1 Memory map (DLAB = 0)

Base address is configurable per instance by the platform integrator (it maps to `UART_REG_MAP_BASE_ADDR` in the `uart_wrap` RTL, which spaces instances by `0x400`).

Offset	Register	Access	Description
0x00	RBR	RO	Receiver Buffer Register — read from RX FIFO / RBR
0x00	THR	WO	Transmitter Holding Register — write to TX FIFO / THR
0x04	IER	RW	Interrupt Enable Register
0x08	IIR	RO	Interrupt Identification Register
0x08	FCR	WO	FIFO Control Register (same address as IIR)
0x0C	LCR	RW	Line Control Register
0x10	MCR	RW	Modem Control Register
0x14	LSR	RO	Line Status Register
0x18	MSR	RO	Modem Status Register
0x1C	SCR	RW	Scratch Register
0x20	ECR	RW	Extended Control Register (RX FIFO trigger level MS-2-bits)
0x24	ITR	RW	Interrupt Test Register

7.2 Memory map (DLAB = 1, LCR[7] = 1)

Offset	Register	Access	Description
0x00	DLL	RW	Divisor Latch LSB
0x04	DLM	RW	Divisor Latch MSB
0x08..0x24	(same as DLAB=0)	—	All other registers unaffected by DLAB

7.3 Detailed register descriptions

RBR — Receiver Buffer Register (0x00, RO, DLAB=0)

Bits	Field	Description
7:0	DATA	Received character. Reading pops the bottom of the RX FIFO (FIFO mode) or the RBR (non-FIFO mode). Reading when empty returns 0x00.
31:8	—	Reserved, reads 0

THR — Transmitter Holding Register (0x00, WO, DLAB=0)

Bits	Field	Description
7:0	DATA	Write to push a byte into the TX FIFO (FIFO mode) or into the single-byte THR (non-FIFO mode). Write is silently dropped if the FIFO is full.

IER — Interrupt Enable Register (0x04, RW)

Bits	Field	Reset	Description
0	ERBFI	0	Enable Received-Data-Ready interrupt
1	ETBEI	0	Enable Transmitter-Holding-Register-Empty interrupt
2	ELSI	0	Enable Receiver-Line-Status interrupt (OE/PE/FE/BI)
3	EDSSI	0	Enable Modem-Status interrupt (DCTS/DDSR/TERI/DDCD)
4	EFEI	0	Enable FIFO-Error interrupt
31:5	—	0	Reserved

IIR — Interrupt Identification Register (0x08, RO)

Bits	Field	Reset	Description
0	INTERRUPT_PENDING	1	Active-low. 0 = interrupt pending, 1 = no interrupt
3:1	INTERRUPT_ID	0	Highest-priority pending interrupt ID (see §6.6)
5:4	—	0	Reserved
7:6	FIFOS_ENABLED	0	2'b11 when FIFO mode active, 2'b00 otherwise

Reading IIR clears the Received-Data-Ready interrupt (if it was the pending source).

FCR — FIFO Control Register (0x08, WO)

Bits	Field	Reset	Description
0	FIFO_ENABLE	0	Enable TX and RX FIFOs. Changing this bit resets both FIFOs and the RBR/THR.
1	RCVR_FIFO_RESET	0	Self-clearing: resets and clears the RX FIFO
2	XMIT_FIFO_RESET	0	Self-clearing: resets and clears the TX FIFO
3	DMA_MODE_SELECT	0	0 = DMA Mode 0; 1 = DMA Mode 1
5:4	—	0	Reserved
7:6	RCVR_TRIGGER	0	RX FIFO trigger level (LS-2-bits; combined with ECR.RCVR_TRIGGER_MS2B)

LCR — Line Control Register (0x0C, RW)

Bits	Field	Reset	Description
1:0	WLS	0	Word length: 0=5-bit, 1=6-bit, 2=7-bit, 3=8-bit
2	STB	0	Stop bits: 0=1 stop, 1=2 stop (1.5 for WLS=0)
3	PEN	0	Parity enable
4	EPS	0	Even parity select (0=odd, 1=even)
5	STICK_PARITY	0	Stick parity: forces parity bit to fixed value
6	SET_BREAK	0	Forces tx_o to 0
7	DLAB	0	Divisor Latch Access Bit

MCR — Modem Control Register (0x10, RW)

Bits	Field	Reset	Description
0	DTR	0	Data Terminal Ready; drives dtr_no = ~DTR
1	RTS	0	Request to Send; drives rts_no = ~RTS
2	OUT1	0	User output 1; drives out1_no = ~OUT1
3	OUT2	0	User output 2; drives out2_no = ~OUT2
4	LOOP	0	System loopback
5	LINE_LOOPBACK	0	Line loopback

LSR — Line Status Register (0x14, RO)

Bits	Field	Reset	Description
0	DR	0	Data Ready — RX FIFO (or RBR) non-empty
1	OE	0	Overrun Error — character dropped (sticky, cleared on read)
2	PE	0	Parity Error on top-of-FIFO character (sticky, cleared on read)
3	FE	0	Framing Error on top-of-FIFO character (sticky, cleared on read)
4	BI	0	Break Interrupt — rx_i held low for full frame (sticky, cleared on read)
5	THRE	1	TX FIFO (or THR) empty
6	TEMT	1	TX FIFO and shift register both empty
7	ERROR_IN_RCVR_FIFO	0	Any PE/FE/BI in current FIFO contents

MSR — Modem Status Register (0x18, RO)

Bits	Field	Reset	Description
0	DCTS	0	Delta CTS — CTS changed since last MSR read (sticky, cleared on read)
1	DDSR	0	Delta DSR (sticky, cleared on read)
2	TERI	0	Trailing Edge RI — RI went 1 → 0 (sticky, cleared on read)
3	DDCD	0	Delta DCD (sticky, cleared on read)
4	CTS	0	Current CTS level (active-high inversion of cts_ni)
5	DSR	0	Current DSR level
6	RI	0	Current RI level
7	DCD	0	Current DCD level

In system loopback mode MSR bits 4-7 reflect MCR bits 1-4. In line loopback mode MSR bits 4-7 are 0.

SCR — Scratch Register (0x1C, RW)

Bits	Field	Reset	Description
7:0	SCR	0	Read/write scratchpad. No hardware function.

ECR — Extended Control Register (0x20, RW)

Bits	Field	Reset	Description
1:0	RCVR_TRIGGER_MS2B	0	RX FIFO trigger level most-significant 2 bits. Concatenated with FCR.RCVR_TRIGGER[1:0] to form a 4-bit trigger level selector (0..11 valid, see table below).

RX FIFO trigger level table (4-bit value = {ECR.RCVR_TRIGGER_MS2B, FCR.RCVR_TRIGGER}):

Value	Trigger (characters)
0x0	1
0x1	4
0x2	8
0x3	14
0x4	32
0x5	64
0x6	128
0x7	256
0x8	512
0x9	1024
0xA	2048
0xB	4096

ITR — Interrupt Test Register (0x24, RW)

Bits	Field	Reset	Description
0	TRBFI	0	Force Received-Data-Ready interrupt
1	TTBEI	0	Force THRE interrupt
2	TLSI	0	Force Receiver-Line-Status interrupt
3	TDSSI	0	Force Modem-Status interrupt
4	TFEI	0	Force FIFO-Error interrupt
5	TRTI	0	Force Reception-Timeout interrupt

DLL — Divisor Latch LSB (0x00, RW, DLAB=1)

Bits	Field	Reset	Description
7:0	DLL	0	Baud divisor bits [7:0]. Setting divisor to 0 disables TX and RX.

DLM — Divisor Latch MSB (0x04, RW, DLAB=1)

Bits	Field	Reset	Description
7:0	DLM	0	Baud divisor bits [15:8].

Baud rate formula: $\text{baud_rate} = \text{system_clock} / (16 \times \{\text{DLM}, \text{DLL}\})$

8. Functional behaviour

8.1 Transmit sequence

1. Software writes a character to THR (DLAB=0, offset 0x00).
2. The byte is pushed to the TX FIFO (or held in the THR single-byte register in non-FIFO mode).
3. When the TX shift register is idle and the FIFO is non-empty, the byte is popped and serialised: start bit (0), data bits LSB-first, optional parity bit, stop bit(s).
4. LSR.THRE rises when the TX FIFO (or THR) becomes empty; the THRE interrupt fires if IER.ETBEI = 1.
5. LSR.TEMT rises when both the TX FIFO and the shift register are empty.

8.2 Receive sequence

1. An incoming character arrives on rx_i.
2. The RX engine detects the start bit, samples 16× per bit period (majority vote at cycles 7, 8, 9 of each bit), and assembles the character.
3. On a valid stop bit, rx_valid is pulsed. The character (with per-character error flags) is pushed into the RX FIFO (or written to RBR in non-FIFO mode).
4. LSR.DR rises when the RX FIFO (or RBR) is non-empty.
5. An overflow is flagged (LSR.OE) if a new character arrives when the RX FIFO is full or the RBR already holds an unread character; the incoming character is dropped.
6. Line-status interrupts (LSR.PE, LSR.FE, LSR.BI) are set based on the framing of the received character.

8.3 FIFO reset behaviour

Setting FCR.RCVR_FIFO_RESET clears and resets the RX FIFO (self-clearing bit). Setting FCR.XMIT_FIFO_RESET clears and resets the TX FIFO. Toggling FCR.FIFO_ENABLE resets both FIFOs and the single-byte RBR/THR.

8.4 Loopback modes

Mode	RX input source	tx_o driven by	Modem out	Modem in (MSR)
Normal	rx_i	TX serialiser	~MCR. {DTR,RTS,OUT1,OUT2}	~modem inputs
System (LOOP=1)	transmitted char	forced 1	forced 1 (deasserted)	MCR. {RTS,DTR,OUT1,OUT2}
Line (LINE_LOOPBACK=1)	idle	rx_i	follows modem inputs	forced 0

Line loopback takes precedence over system loopback.

9. Reset behaviour

On assertion of rst_ni = 0 (active-low reset):

- All control registers (IER, FCR, LCR, MCR, ECR, ITR, DLL, DLM, SCR) are cleared to 0.
- Status registers (LSR, MSR) return to their power-on values: LSR = 0x60 (THRE=1, TEMT=1), MSR reflects the actual modem inputs.

- IIR.INTERRUPT_PENDING is set to 1 (no interrupt active).
- TX and RX FIFOs are cleared.
- `tx_o` is driven to 1 (idle).
- All modem outputs (`rts_no`, `dtr_no`, `out1_no`, `out2_no`) are driven to 1 (deasserted, active-low).
- `irq_o`, `rxrdy_o`, `txrdy_o`, `err_o` are driven to 0.

10. Bus interface (TLM-2.0 / AXI4-Lite)

10.1 Transport function

The model exports a single `tlm_utils::simple_target_socket<uart>` named `reg_socket`. The `b_transport` implementation:

1. Validates the generic payload (command must be `TLM_READ_COMMAND` or `TLM_WRITE_COMMAND`; data length must be exactly 4; address must be 4-byte aligned).
2. Adds `access_delay_ns` to the `sc_time& delay` argument (temporal decoupling — quantum keeper interaction is in the test bench / initiator).
3. Dispatches to `reg_read()` or `reg_write()` which perform the DLAB mux and all register side effects.
4. Returns `TLM_OK_RESPONSE` on success.

10.2 Error responses

Condition	Response code
Out-of-window address ($\geq$ WINDOW_SIZE)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
Unaligned address (<code>addr & 0x3 != 0</code>)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
Unsupported data length (not 4)	<code>TLM_BURST_ERROR_RESPONSE</code>
Unsupported command (not READ/WRITE)	<code>TLM_COMMAND_ERROR_RESPONSE</code>

10.3 Debug transport

`transport_dbg` is implemented: it performs the register access with no delay and no side effects (read-only: does not pop the RX FIFO, does not clear sticky bits). Returns the number of bytes transferred (4 on success, 0 on error).

10.4 DMI

DMI is not granted. The register file has read side effects (IIR read clears the pending interrupt; MSR read clears delta bits; RBR read pops the FIFO). Granting DMI would bypass these effects.

11. Error handling

11.1 RX errors

Error	Condition	LSR bit	Interrupt trigger
Overrun Error	New char arrives when FIFO full / RBR occupied	LSR.OE	RLS interrupt (IER.ELSI)
Parity Error	Computed parity != received parity	LSR.PE	RLS interrupt
Framing Error	No valid stop bit detected	LSR.FE	RLS interrupt
Break Interrupt	rx_i held 0 for a full frame	LSR.BI	RLS interrupt
FIFO Error	Any PE/FE/BI in FIFO contents	LSR.ERROR_IN_RCVR_FIFO	FIFO Error interrupt (IER.EFEI)

All error bits in LSR are sticky and cleared on each read of LSR. In FIFO mode, the error bits (OE/PE/FE/BI) reflect the character at the top of the FIFO; they update as characters are popped.

11.2 Aggregate error output

`err_o` is the OR of all active unmasked RX error conditions. It can be used by the fabric error-slave to flag persistent errors without polling.

12. Programming model

12.1 Initialisation sequence

1. Assert reset (`rst_ni = 0`), then de-assert (`rst_ni = 1`).
2. Set DLAB=1 (`LCR = 0x80`).
3. Write DLL and DLM with the desired divisor:
`divisor = system_clock_hz / (16 * baud_rate)`
 Example: 50 MHz clock, 115200 baud → divisor = 27 (0x1B)
`DLL = 0x1B, DLM = 0x00`
4. Clear DLAB, configure line parameters (LCR):
`LCR = 0x03 → 8N1 (8 data, no parity, 1 stop)`
`LCR = 0x07 → 8E2 (8 data, even parity, 2 stop)`
5. Enable and reset FIFOs (FCR):
`FCR = 0x07 → enable FIFOs, reset TX+RX, DMA Mode 0`
6. Enable desired interrupts (IER):
`IER = 0x01 → received-data-ready only`
`IER = 0x0F → all standard interrupts`
7. Configure modem outputs if needed (MCR):
`MCR = 0x03 → assert DTR and RTS`

12.2 Transmit a character

```
while (!(lsr & 0x20)) // poll LSR.THRE
    lsr = read(UART_BASE + 0x14);
write(UART_BASE + 0x00, character); // write to THR
```

12.3 Receive a character

```
while (!(lsr & 0x01)) // poll LSR.DR
    lsr = read(UART_BASE + 0x14);
ch = read(UART_BASE + 0x00); // read from RBR
if (lsr & 0x1E)
    handle_rx_error(lsr); // OE | PE | FE | BI
```

12.4 Interrupt-driven operation

ISR entry:

1. Read IIR (offset 0x08).
2. If IIR[0]=1, no interrupt – spurious call, return.
3. Switch on IIR[3:1]:
 - 0x7 → FIFO Error: read LSR to clear ERROR_IN_RCVR_FIFO
 - 0x3 → Line Status: read LSR to clear OE/PE/FE/BI
 - 0x6 → RX Timeout: drain RX FIFO until LSR.DR=0
 - 0x2 → Data Ready: read characters until LSR.DR=0 (or trigger threshold)
 - 0x1 → THRE: fill TX FIFO up to tx_fifo_depth characters
 - 0x0 → Modem Stat: read MSR to clear DCTS/DDSR/TERI/DDCD
4. Re-read IIR; repeat if IIR[0] still 0 (multiple pending sources).

13. Compliance matrix

Feature	RTL	SystemC model	Notes
All registers (RBR, THR, IER, IIR, FCR, LCR, MCR, LSR, MSR, SCR, ECR, ITR, DLL, DLM)	✓	✓	Full register file modelled, bit-compatible with the RDL
DLAB muxing at 0x00 and 0x04	✓	✓	
FIFO enable/reset/DLAB-toggle reset	✓	✓	
RX FIFO 4-bit trigger level (FCR+ECR)	✓	✓	
6-source interrupt priority chain	✓	✓	Fixed-priority encode into IIR.INTERRUPT_ID
IIR read clears data-ready interrupt	✓	✓	
MSR delta bits cleared on read	✓	✓	
System loopback	✓	✓	
Line loopback	✓	✓	
Modem control outputs	✓	✓	
Modem status delta detection	✓	✓	
DMA Mode 0	✓	✓	
DMA Mode 1 (TX/RX FSMs)	✓	✓	
Parity computation on TX	✓	✓	
RX framing/parity/break error	✓	✓	Errors injected by test bench through <code>inject_rx_char()</code>
<code>err_o</code> aggregate	✓	✓	
ITR force-interrupt bits	✓	✓	
<code>transport_dbg</code> no-side-effect read	n/a	✓	Model-only debug path
Bit-accurate baud-rate timing	✓	NOT modelled	Functional/LT model: TX/RX at character granularity with annotated delay, not bit-by-bit timing
3-sample majority filter on <code>rx_i</code>	✓	NOT modelled	Noise filtering not meaningful at the functional abstraction
2-FF synchroniser on modem inputs	✓	NOT modelled	Clock-domain crossing not relevant in an LT model

14. Revision history

Revision	Date	Author	Notes
0.1	2026-06-23	SMC team	Initial draft; tracks <code>uart_16550</code> RTL and the <code>uart_16550_*.rdl</code> register maps

15. Glossary

Term	Definition
DLAB	Divisor Latch Access Bit. Set in LCR[7] to access DLL/DLM.
DLL/DLM	Divisor Latch LSB/MSB. Together they form the 16-bit baud-rate divisor.
THR	Transmitter Holding Register. Write port to the TX FIFO.
RBR	Receiver Buffer Register. Read port from the RX FIFO.
FCR	FIFO Control Register. Enables FIFOs and sets their trigger levels.
IIR	Interrupt Identification Register. Reports the highest-priority pending interrupt.
LCR	Line Control Register. Configures word format and baud divisor access.
LSR	Line Status Register. Reports TX/RX status and error flags.
MSR	Modem Status Register. Reports modem input levels and delta bits.
FIFO mode	Operating mode where FCR.FIFO_ENABLE = 1 and 16-byte (or deeper) FIFOs are active.
LT	Loosely-Timed. TLM-2.0 modelling style using <code>b_transport</code> with annotated delay.
QK	Quantum Keeper (<code>tlm_utils::tlm_quantumkeeper</code>). Manages temporal decoupling in LT models.
baud_x16	Internal clock tick at 16× baud rate used for sampling and serialisation.
THRE	Transmitter Holding Register Empty — TX FIFO drained, ready for more data.
TEMT	Transmitter Empty — TX FIFO and shift register both empty; line is idle.